

LMS Platform — Backend Gap Analysis Report

Project: Online Course Platform
Date: 2026-07-07
Prepared by: Gihan (Backend Developer)
Repository: <https://github.com/Lahiru-2001/Online-Course-Platform.git>
Branch: gihan-backend / test

1. Executive Summary

Backend structure (Node.js + Express.js) has been fully scaffolded with routes, controllers, models, and middlewares. However, a detailed comparison between the frontend UI and the backend reveals that the majority of controllers contain only placeholder logic and several critical API endpoints required by the frontend are either missing or not fully implemented.

2. Current Backend Status (What is Done)

Completed Items

Category	Status
Folder structure (config, controllers, middlewares, models, routes)	Complete
Express.js server setup (server.js, app.js)	Complete
MongoDB connection (db.js)	Complete
Environment variables (.env)	Complete
Auth routes (register, login, forgot-password)	Complete
Auth controller with JWT token generation	Complete
authMiddleware.js (JWT verification)	Complete
roleMiddleware.js (role-based access)	Complete
validationMiddleware.js	Complete
Course controller (full CRUD)	Complete
User controller (CRUD)	Complete
All route files registered in app.js	Complete
User model (with bcrypt password hashing)	Complete
Course model	Complete
Enrollment model (with duplicate prevention index)	Complete
Assignment, Quiz, ForumPost, Notification models	Complete
Certificate, Payment, Progress, Lesson, ChatMessage models	Complete

3. Critical Gaps (Must Fix)

3.1 Placeholder Controllers — No Real Database Logic

The following controllers return only static messages and do not interact with MongoDB:

Controller File	Placeholder Functions
assignmentController.js	getAssignments, getAssignmentById, createAssignment, updateAssignment, deleteAssignment
enrollmentController.js	getEnrollments, getEnrollmentById, createEnrollment, updateEnrollment, deleteEnrollment
quizController.js	getQuizzes, getQuizById, createQuiz, updateQuiz, deleteQuiz
chatController.js	getMessages, sendMessage
forumController.js	getPosts, createPost, getPostById, updatePost, deletePost
notificationController.js	getNotifications, markAsRead, deleteNotification
progressController.js	updateProgress, getProgress
paymentController.js	processPayment, getPaymentHistory (mock only)
certificateController.js	generateCertificate (mock URL, no real PDF)
lessonController.js	getLessons, uploadLearningMaterials (no Multer config)

Impact: All frontend pages that expect real data from these endpoints will receive only static messages.

3.2 Routes Not Protected with authMiddleware

All routes are currently unprotected. Any user (authenticated or not) can call any endpoint.

Required: Apply authMiddleware and roleMiddleware to the following:

Route	Required Access
POST /api/courses	Instructor only
PUT /api/courses/:id	Instructor (own course) or Admin
DELETE /api/courses/:id	Instructor (own course) or Admin
GET /api/users	Admin only
DELETE /api/users/:id	Admin only
POST /api/enrollments	Student only
GET /api/progress/:courseId	Authenticated Student
POST /api/certificates/generate	Authenticated Student
GET /api/payments/history	Authenticated Student
GET /api/notifications	Authenticated User

3.3 Missing API Endpoints (Frontend Expects These)

The following endpoints are required by the frontend but do not exist:

Authentication and User Management

- PUT /api/users/:id/approve — Admin approves instructor application
- PUT /api/users/:id/reject — Admin rejects instructor application
- GET /api/users/stats — Dashboard summary stats
- PUT /api/users/profile — Update user profile

Enrollment and Learning

- POST /api/enrollments — Student enrolls in a course (real logic)
- GET /api/enrollments/my — Get student enrolled courses
- PUT /api/progress/lesson — Mark a specific lesson as completed
- GET /api/progress/my — Get all progress for logged-in student

Courses and Lessons

- GET /api/courses/my — Get instructor own courses
- POST /api/lessons — Create a lesson under a course
- GET /api/lessons/course/:courseId — Get all lessons for a course
- POST /api/lessons/:id/upload — Upload material (PDF/MP4/ZIP) needs Multer

Quiz and Assignment

- POST /api/quizzes/:id/submit — Student submits quiz answers
- GET /api/quizzes/course/:courseId — Get quizzes for a course
- POST /api/assignments/:id/submit — Student submits assignment
- GET /api/assignments/course/:id — Get assignments for a course

Reports and Analytics (Admin)

- GET /api/reports/analytics — Platform-wide statistics for admin
- GET /api/reports/revenue — Revenue report (monthly breakdown)
- GET /api/reports/enrollments — Enrollment trends

4. Important Gaps

4.1 File Upload System Not Configured

- Frontend UploadMaterial.jsx allows uploading PDF, MP4, ZIP, PPTX files (max 500MB)
- Backend has no Multer middleware configured
- No cloud storage (Cloudinary or AWS S3) integration exists

Fix Required:

npm install multer cloudinary multer-storage-cloudinary

Then configure uploadMiddleware.js and apply to lesson upload routes.

4.2 Email System Not Configured (Forgot Password)

- Frontend ForgotPassword.jsx exists and calls the endpoint
- Backend forgotPassword controller only returns a success message — no actual email is sent

Fix Required:

npm install nodemailer

Configure SMTP in .env and implement password reset token logic.

4.3 Certificate PDF Generation Not Implemented

- Frontend Certificates.jsx shows certificate download buttons
- Backend certificateController.js returns a mock URL only

Fix Required:

npm install pdfkit

Implement real PDF generation with student name, course name, and completion date.

4.4 Frontend Not Connected to Backend

This is the most critical integration gap:

Item	Status
services/api.js	Empty file (0 bytes) — no Axios/fetch setup
Login.jsx	Uses password string comparison only — no API call
Register.jsx	Navigates by password string — no API call
AuthContext.jsx	localStorage only — no JWT token validation
All data pages	100% hardcoded data — no backend calls

5. Missing npm Packages

The following packages need to be installed in the backend:

- multer — File uploads
- nodemailer — Email sending
- pdfkit — PDF generation
- helmet — Security headers
- express-rate-limit — Rate limiting (prevent brute force)
- express-validator — Input validation
- morgan — HTTP request logger

6. Summary Table — All Gaps

Priority	Gap	Area
CRITICAL	Placeholder controllers (10 files)	Controllers
CRITICAL	Routes not protected (authMiddleware not applied)	Middlewares
CRITICAL	Frontend not connected to backend (api.js empty)	Integration
CRITICAL	Missing enrollment real logic	Controllers
CRITICAL	Missing quiz submission endpoint	Routes + Controllers
IMPORTANT	File upload (Multer) not configured	Middlewares
IMPORTANT	Forgot password email not implemented	Controllers
IMPORTANT	Certificate PDF generation not implemented	Controllers
IMPORTANT	Instructor approval endpoints missing	Routes + Controllers
IMPORTANT	Admin analytics/reports endpoints missing	Routes + Controllers
IMPORTANT	Get my courses / Get my enrollments missing	Routes
IMPORTANT	Lesson CRUD endpoints not fully functional	Controllers
MINOR	Morgan / Helmet security packages missing	Packages
MINOR	Rate limiting not implemented	Middlewares
MINOR	Central error handler middleware missing	Middlewares

7. Recommended Next Steps (Priority Order)

1. Connect frontend to backend — implement services/api.js with Axios
2. Implement real enrollment controller — enroll student, prevent duplicates
3. Apply authMiddleware to all protected routes
4. Implement quiz submit endpoint with answer validation
5. Configure Multer for lesson material upload
6. Implement nodemailer for forgot password flow
7. Add missing endpoints (instructor approval, my-courses, analytics)
8. Implement certificate PDF generation using pdfkit
9. Add security packages (helmet, rate-limit, morgan)